

Modül 8: Performans Optimizasyonu

Büyük veri setlerinde verimsiz bir sorgu yazmak, sadece zaman kaybı değil aynı zamanda ciddi bir hesaplama maliyeti (compute cost) demektir.

8.1. İndeksleme Mantığı (Indexing)

İndeksler, veritabanı motorunun veriyi tüm tabloyu taramadan (Full Table Scan) hızlıca bulmasını sağlar.

- **B-Tree Index:** En yaygın tiptir. Veriyi sıralı bir ağaç yapısında tutar.
- **Ne zaman kullanılır?** WHERE, JOIN ve ORDER BY kısımlarında sıkça kullanılan sütunlara indeks eklenmelidir.

Kod Örneği

```
-- Sık sorgulanan 'musteri_id' sütununa  
-- indeks ekleyerek hızı artırma  
CREATE INDEX idx_musteri_id  
ON satislar (musteri_id);
```

8.2. SELECT * Kullanımından Kaçınma

Veri biliminde genellikle onlarca sütun arasından sadece birkaçına ihtiyaç duyarız. Tüm sütunları çağırmak ağ trafiğini ve bellek (RAM) kullanımını gereksiz yere artırır.

Kod Örneği

```
-- YAVAŞ: Gereksiz tüm sütunları çeker  
SELECT * FROM dev_data_table;  
  
-- HIZLI: Sadece gerekli özellikleri çeker  
SELECT  
    user_id,  
    transaction_amount,  
    event_date  
FROM dev_data_table;
```

8.3. SARGABLE (Search Argumentable) Sorgular

İndeksli bir sütun üzerinde fonksiyon kullanmak, o indeksi devre dışı bırakır ve sorguyu yavaşlatır. Filtreleri sütunun kendisine uygulamak her zaman daha verimlidir.

Kod Örneği

```
-- YAVAŞ: İndeksi bozar (Sütun üzerinde fonksiyon var)
SELECT * FROM satislar
WHERE YEAR(satis_tarihi) = 2025;

-- HIZLI: İndeksi kullanır (Sütun yalın halde)
SELECT * FROM satislar
WHERE satis_tarihi >= '2025-01-01'
AND satis_tarihi <= '2025-12-31';
```

8.4. EXPLAIN / EXPLAIN ANALYZE

Bir sorguyu çalıştırmadan önce veritabanının hangi yolları izleyeceğini görmek için kullanılır. Darboğazları (bottlenecks) tespit etmenin en profesyonel yoludur.

Kod Örneği

```
-- Sorgu maliyetini ve indeks
-- kullanımını analiz etme
EXPLAIN ANALYZE
SELECT
    kategori,
    AVG(fiyat)
FROM urunler
WHERE stok_durumu = 'Aktif'
GROUP BY kategori;
```

8.5. Alt Sorgular Yerine CTE ve Join Kullanımı

Bazı veritabanı motorlarında IN operatörü ile yazılan alt sorgular yavaş çalışabilir. Bunları JOIN veya CTE yapısına çevirmek genellikle performansı artırır.

Kod Örneği

```
-- OPTİMİZE EDİLMİŞ YAPI (CTE ile)
WITH aktif_musteriler AS (
    SELECT id
    FROM users
    WHERE status = 'Active'
)
SELECT
    s.tutar,
    s.tarih
FROM satislar s
JOIN aktif_musteriler am
    ON s.user_id = am.id;
```